



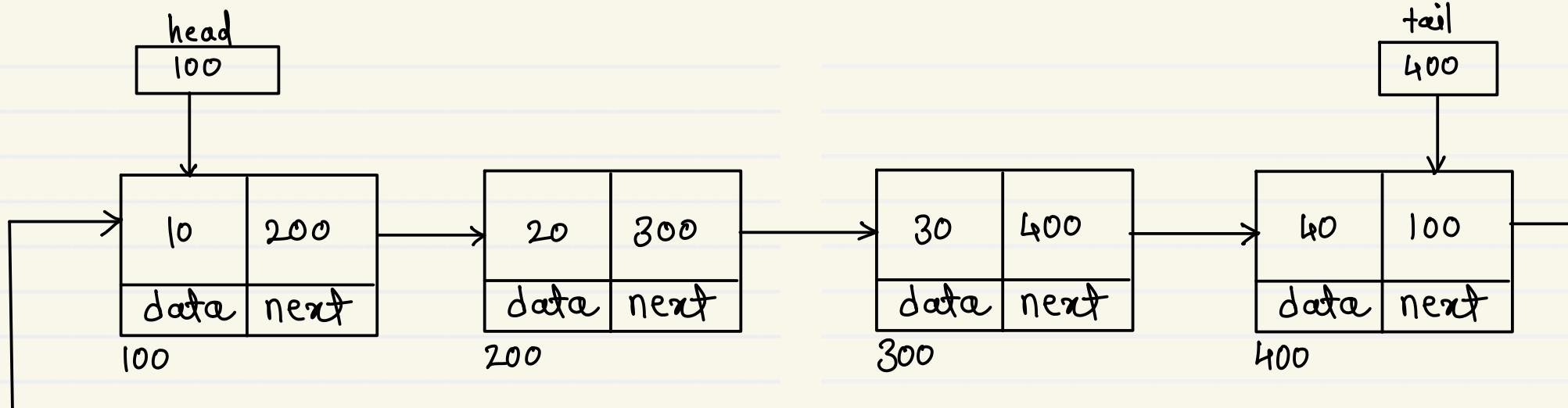
Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Singly Circular Linked List - Display



1. create trav & start at first node
2. visit / print current data
3. go on next node
4. repeat above two steps till last node

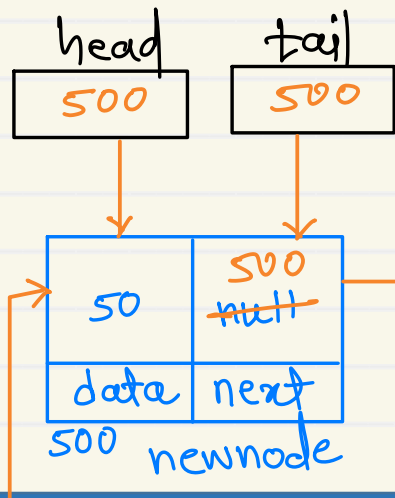
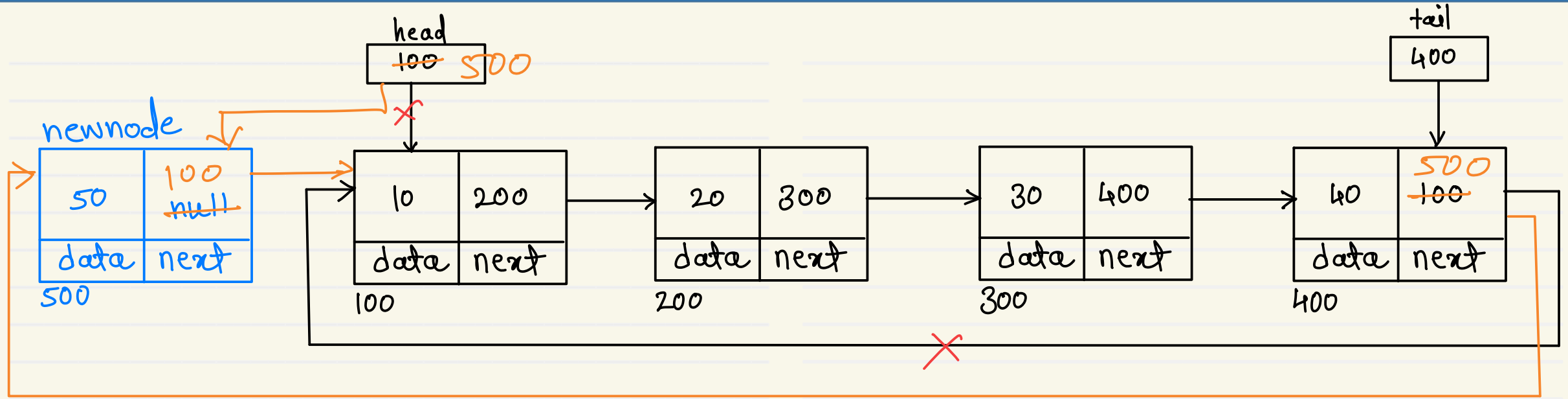
$$T(n) = O(n)$$

```

Node trav = head;
do {
    sysout ( trav.data )
    trav = trav.next;
} while ( trav != head );
    
```

trav	trav.data
100	10
200	20
300	30
400	40
100	X

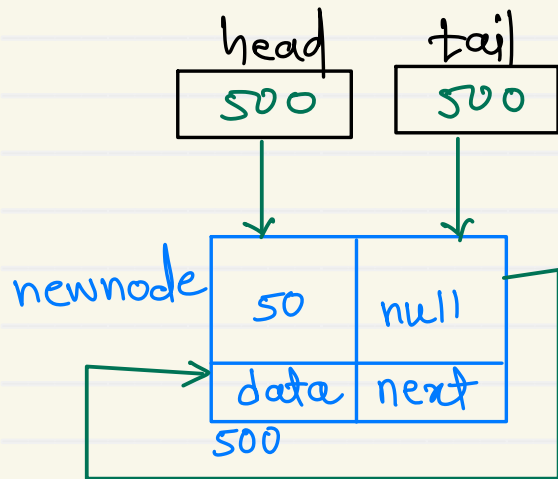
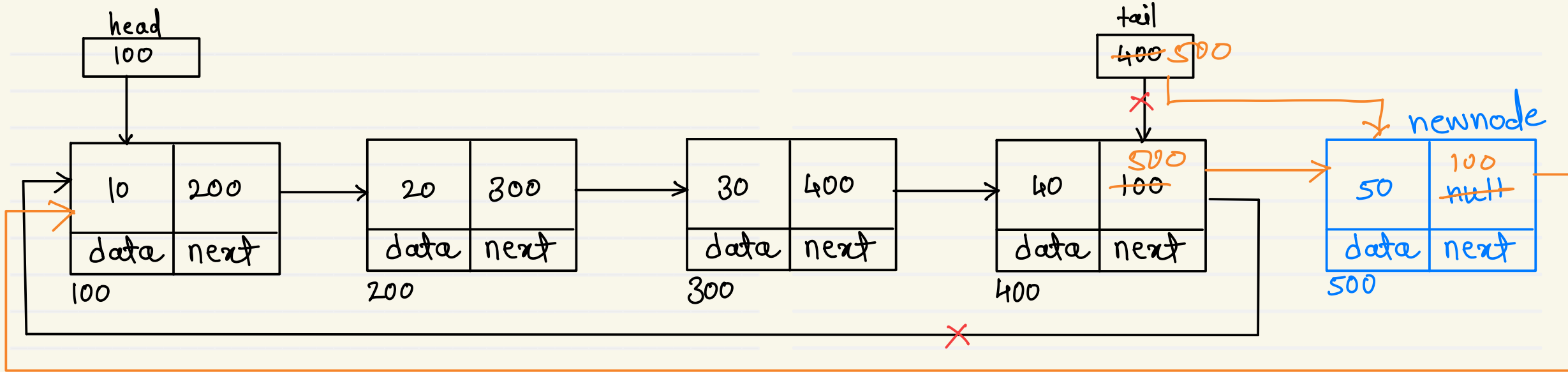
Singly Circular Linked List - Add first



1. Create a newnode
2. if list is empty,
 - a. add newnode into head & tail
 - b. make list circular
3. if list is not empty
 - a. add first node into next of newnode
 - b. add newnode into next of last node
 - c. move head on newnode

$$T(n) = O(1)$$

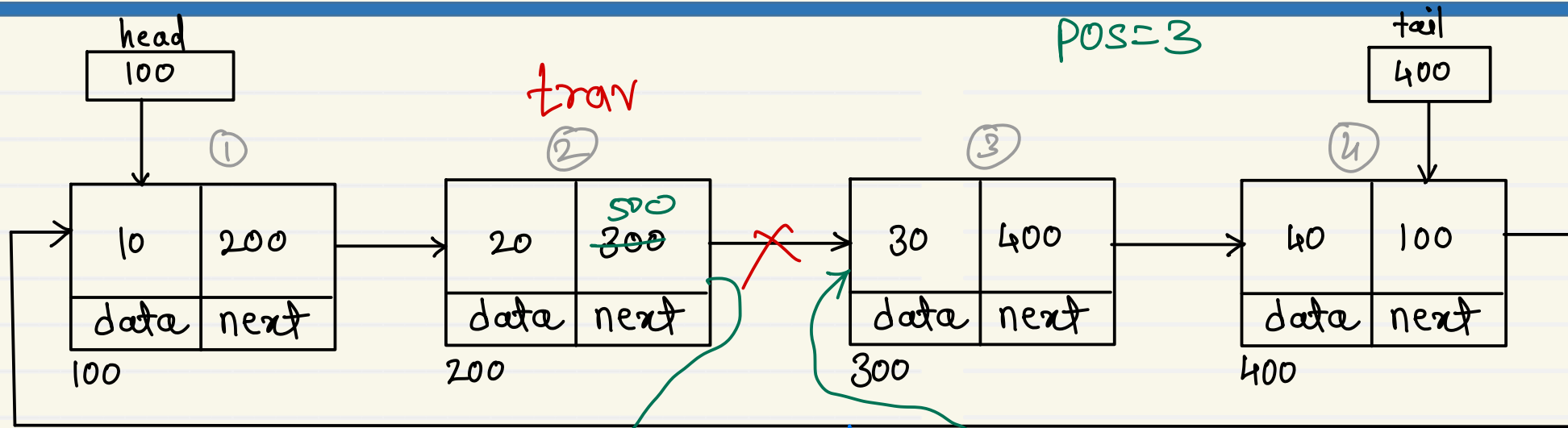
Singly Circular Linked List - Add last



1. Create a newnode
2. if list is empty,
 - a. add newnode into head & tail
 - b. make list circular
3. if list is not empty
 - a. add first node into next of newnode
 - b. add newnode into next of last node
 - c. move tail on newnode

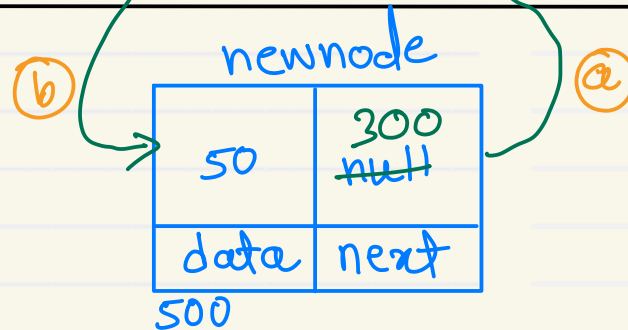
$$T(n) = O(1)$$

Singly Circular Linked List - Add position



valid positions:
1 to size+1
invalid positions:
pos < 1
pos > size+1

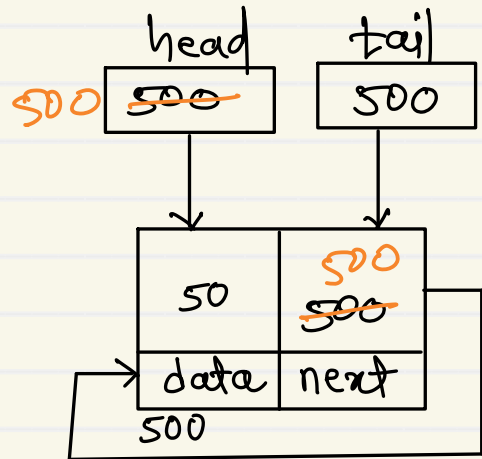
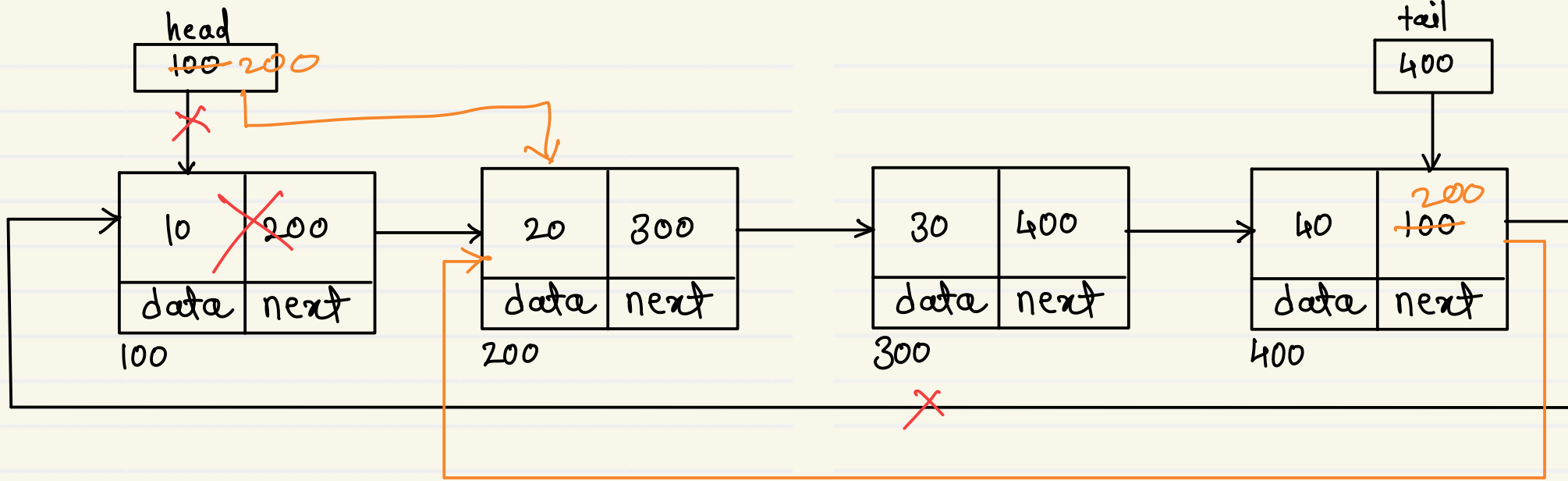
make before break



1. create a newnode
2. if list is empty
add newnode in head & tail.
make list circular.
3. if list is not empty
 - a. traverse till pos-1 node
 - b. add pos node into next of newnode
 - c. add newnode into next of pos-1 node

$$T(n) = O(n)$$

Singly Circular Linked List - Delete first

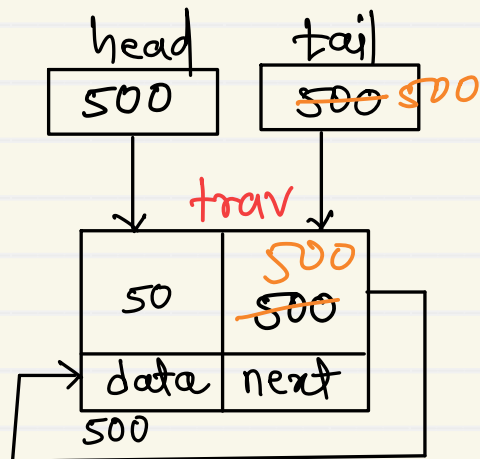
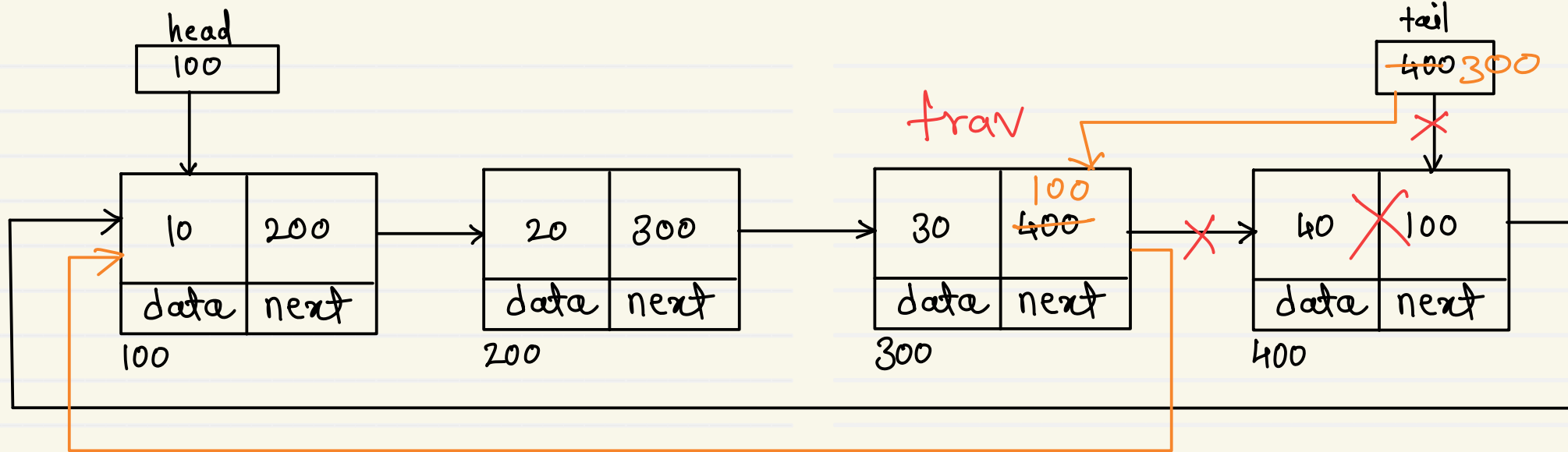


head = head.next;
tail.next = head;

1. if list is empty, return
2. if list has single node,
head = tail = null
3. if list has multiple nodes
 - a. move head on second node
 - b. update next of last node by new head

$$T(n) = O(1)$$

Singly Circular Linked List - Delete last



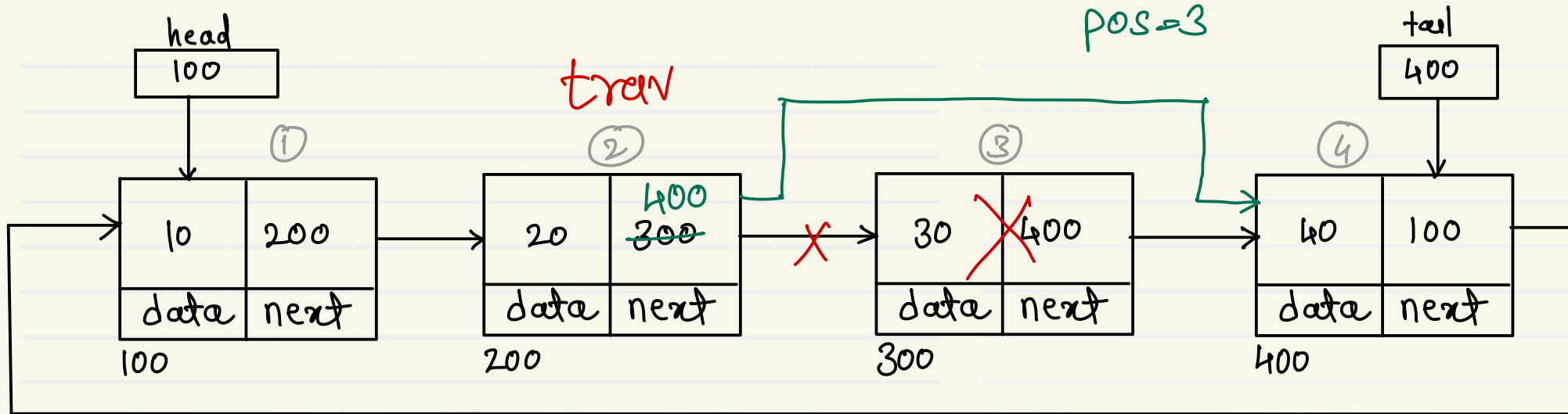
while (trav.next.next != head)
trav = trav.next;

while (trav.next != tail)
trav = trav.next;

$$T(n) = O(n)$$

1. if list is empty, return
2. if list has single node,
head = tail = null
3. if list has multiple nodes
 - a. traverse till second last node
 - b. move tail on second last node
 - c. update next of second last node by head

Singly Circular Linked List - Delete position

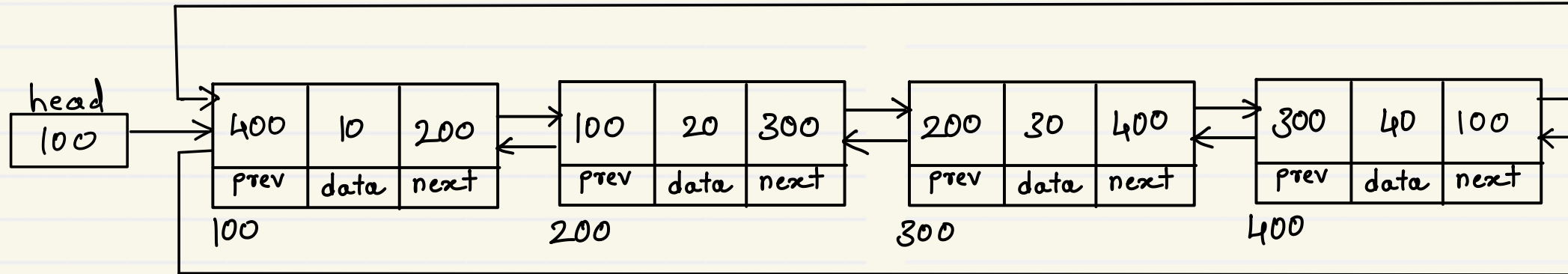


valid positions:
1 to size
invalid positions:
pos < 1
pos > size

1. if list is empty
return
2. if list has single node
head = tail = null.
3. if list has multiple nodes
 - a. traverse till pos-1 node
 - b. add pos+1 node into next of pos-1 node

$$T(n) = O(n)$$

Doubly Circular Linked List - Display

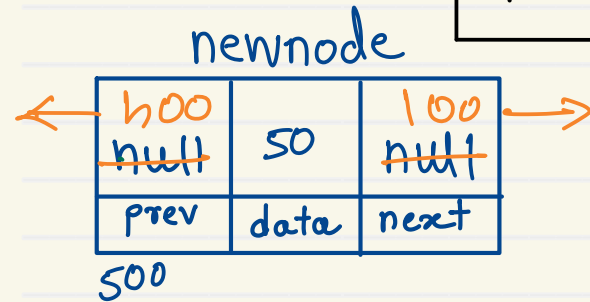
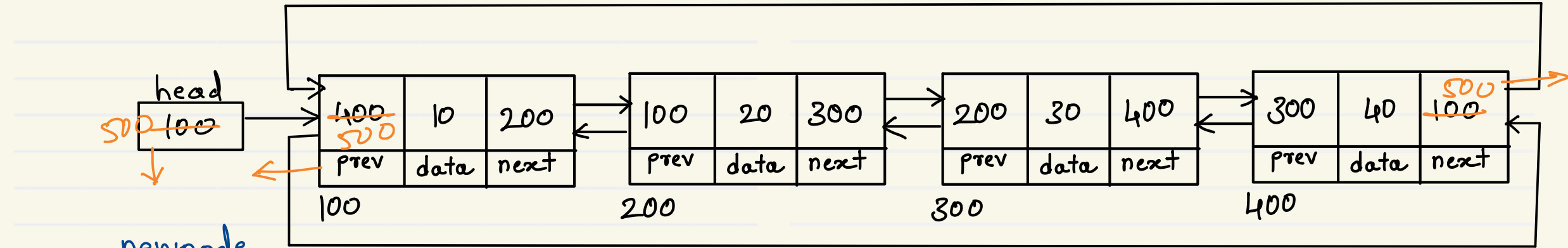


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

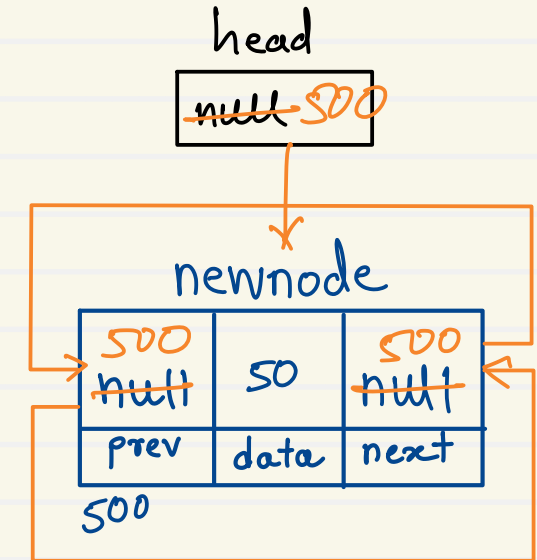
$$T(n) = O(n)$$

Doubly Circular Linked List - Add first

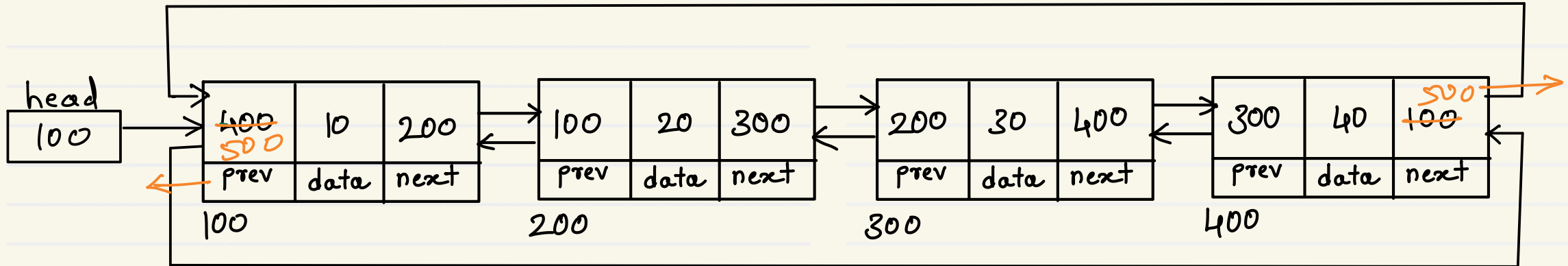


1. Create a newnode
2. if list is empty
 - a. add newnode into head
 - b. make list circular
3. if list is not empty
 - a. add first node into next of newnode
 - b. add last node into prev of newnode
 - c. add newnode into next of last node
 - d. add newnode into prev of first node
 - e. move head on newnode

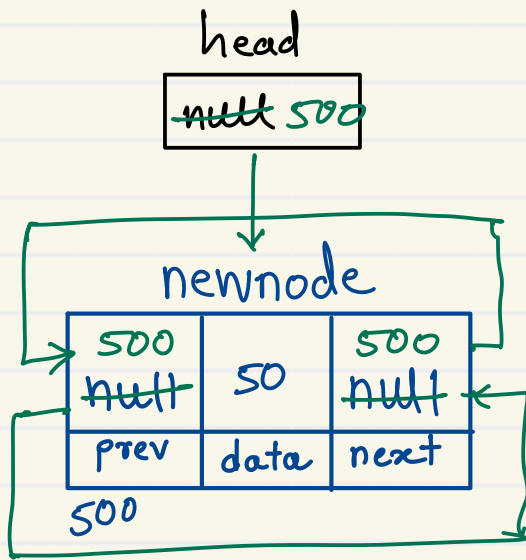
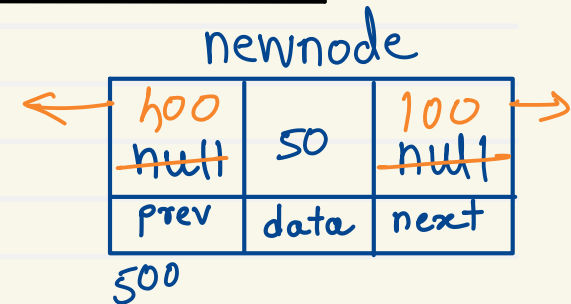
$$T(n) = O(1)$$



Doubly Circular Linked List - Add last

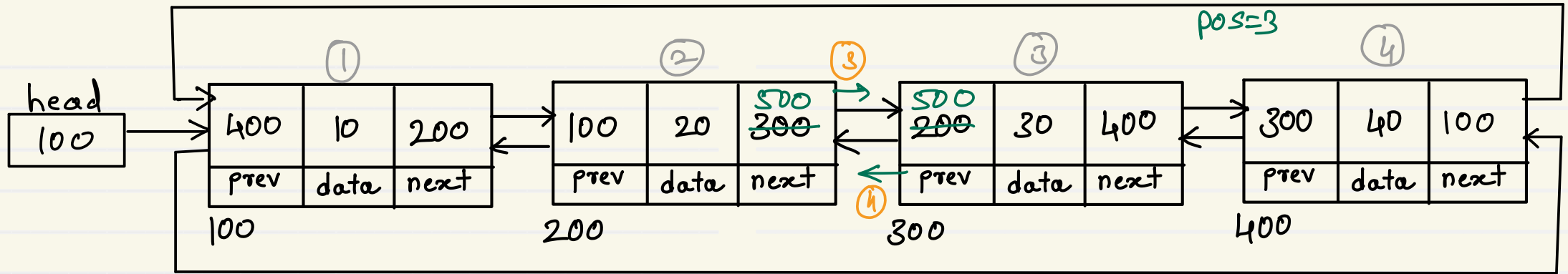


1. Create a newnode
2. if list is empty
 - a. add newnode into head
 - b. make list circular
3. if list is not empty
 - a. add first node into next of newnode
 - b. add last node into prev of newnode
 - c. add newnode into next of last node
 - d. add newnode into prev of first node
 - e. move head on newnode

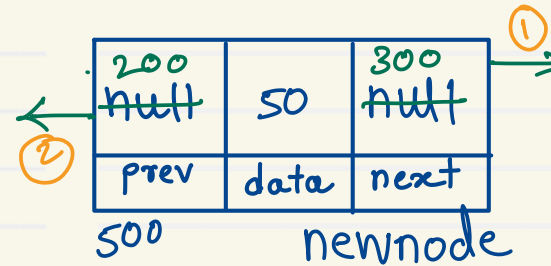


$$T(n) = O(1)$$

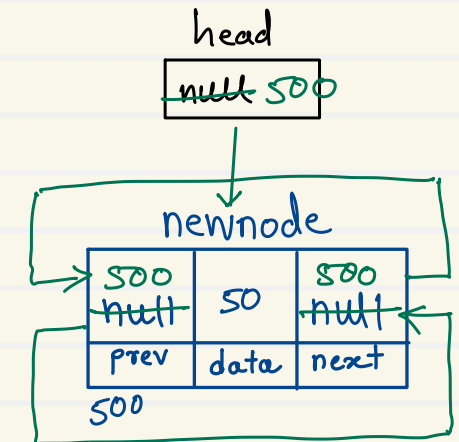
Doubly Circular Linked List - Add position



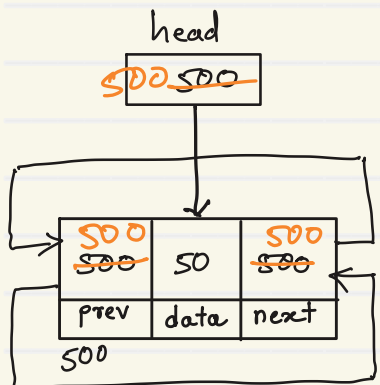
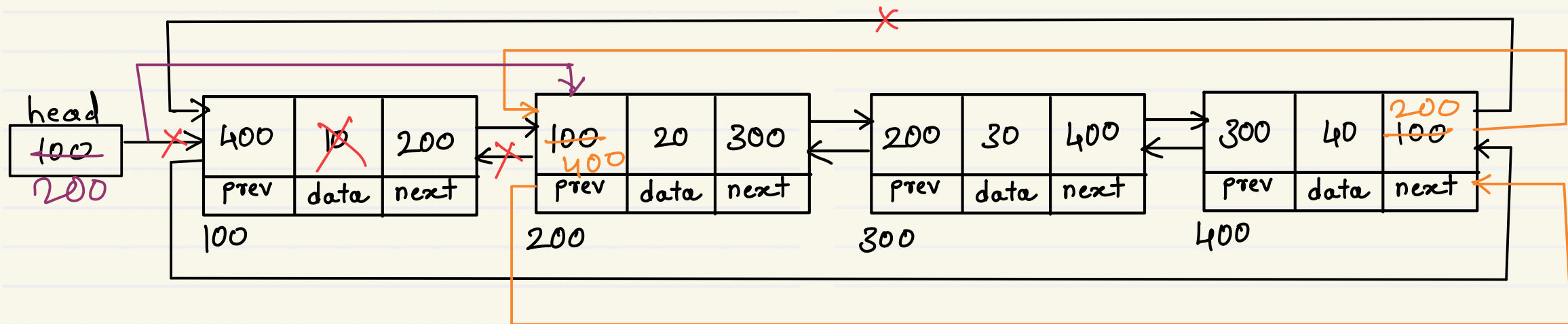
1. create node
2. if list is empty
 - a. add newnode into head
 - b. make list circular
3. if list is not empty.
 - a. traverse till pos-1 node
 - b. add pos node into next of newnode
 - c. add pos-1 node into prev of newnode
 - d. add newnode into next of pos-1 node
 - e. add newnode into prev of pos node



$$T(n) = O(1)$$



Doubly Circular Linked List - Delete first

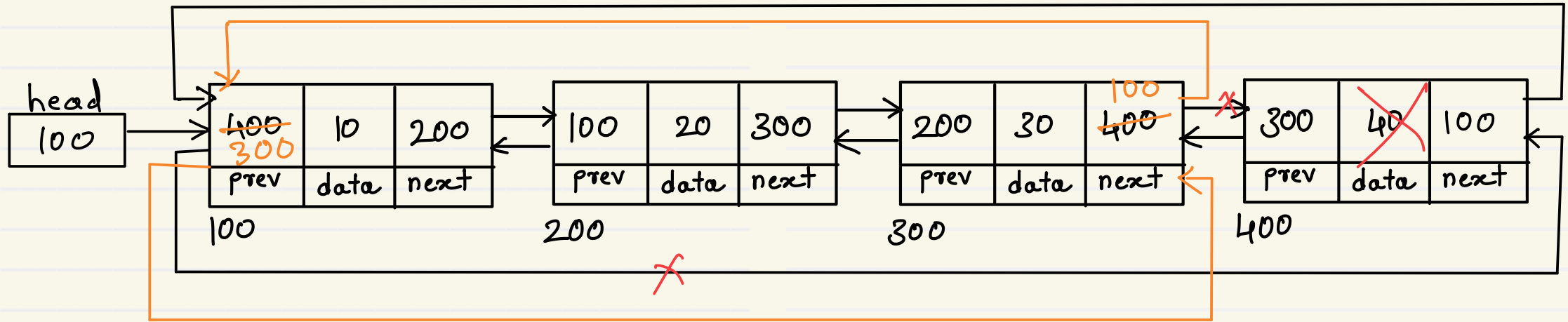


$\text{head}.\text{prev}.\text{next} = \text{head}.\text{next}$
 $\text{head}.\text{next}.\text{prev} = \text{head}.\text{prev}$
 $\text{head} = \text{head}.\text{next}$

$$T(n) = O(1)$$

1. if list is empty, return
2. if list has single node
head = null
3. if list has multiple nodes
 - a. add second node into next of last node
 - b. add last node into prev of second node
 - c. move head on second node

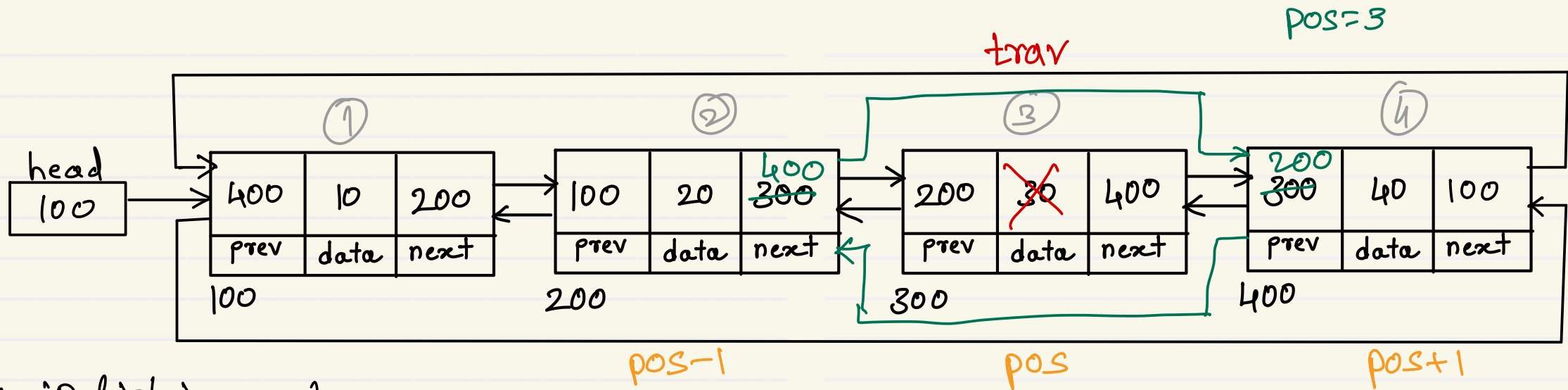
Doubly Circular Linked List - Delete last



1. if list is empty, return
2. if list has single node
head = null
3. if list has multiple nodes
 - a. add second last node into prev of first node
 - b. add first node into next of second last node

$$T(n) = O(1)$$

Doubly Circular Linked List - Delete position



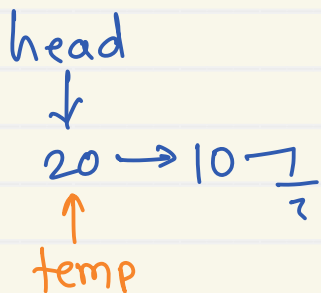
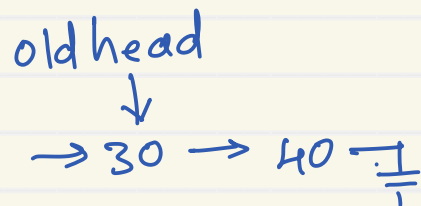
1. if list is empty
return;
2. if list has single node
head = tail = null;
3. if list has multiple nodes
 - a. traverse till pos node
 - b. add pos+1 node into next of pos-1 node
 - c. add pos-1 node into prev of pos+1 node

trav → pos
trav.prev → pos-1
trav.next → pos+1

$$T(n) = O(n)$$

Linked List - competitive/interview question

- reverse singly linked list

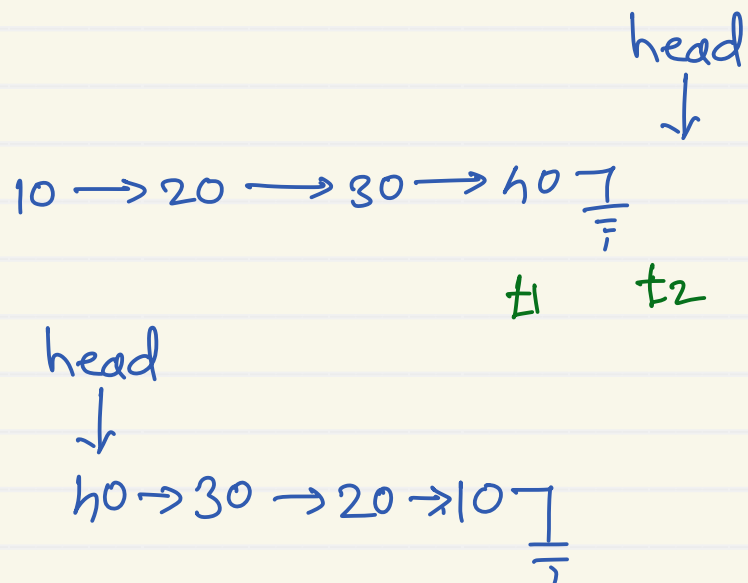


$$T(n) = O(n)$$
$$S(n) = O(1)$$

```
void reverseList( ) {
    // consider current list as old list
    Node oldHead = head;
    // consider new list as empty
    head = null;
    // repeat while old list becomes empty
    while (oldHead != null) {
        // delete first node from old list
        Node temp = oldHead;
        oldHead = oldHead.next;
        // add first node into new list
        temp.next = head;
        head = temp;
    }
}
```

Linked List - competitive/interview question

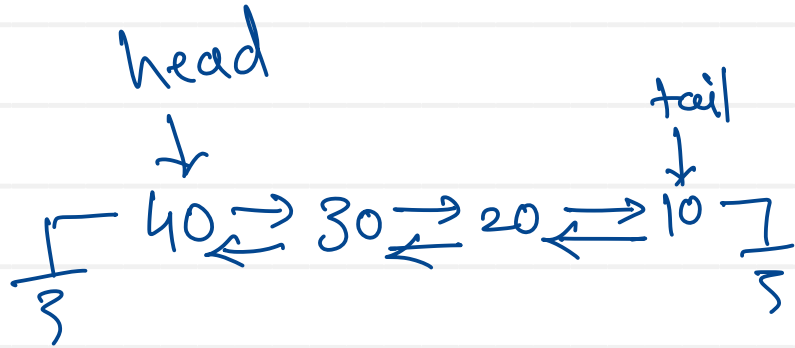
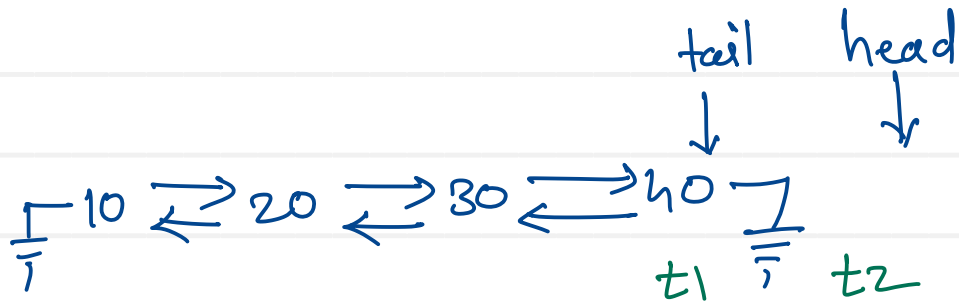
- reverse singly linked list



$$T(n) = O(n)$$
$$S(n) = O(1)$$

```
void reverseList() {  
    Node t1 = null, t2 = head;  
    while (t2 != null) {  
        head = t2.next;  
        t2.next = t1;  
        t1 = t2;  
        t2 = head;  
    }  
    head = t1;  
}
```

Doubly linear linked list - Reverse



$t1.next = t2$
 $t2.prev = t1 \Rightarrow t2.next = t1$
 $t1.prev = t2$

```

Node t1 = head;
Node t2 = head.next;
head.next = null;
tail = head;
while (t2 != null) {
    head = t2.next;
    t2.next = t1;
    t1.prev = t2;
    t1 = t2;
    t2 = head;
}

```

```

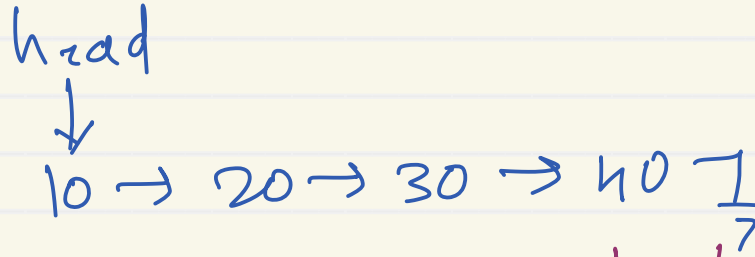
t1.prev = null;
head = t1;

```

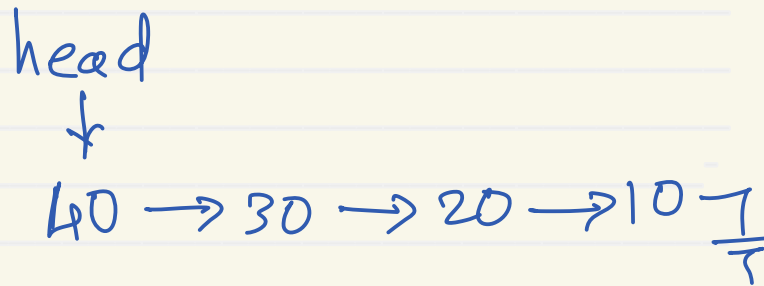
$$T(n) = O(n)$$

$$S(n) = O(1)$$

- reverse singly linked list



	curr	last
recReverseList(10)	10	20
recReverseList(20)	20	30
recReverseList(30)	30	40
recReverseList(40)	40	



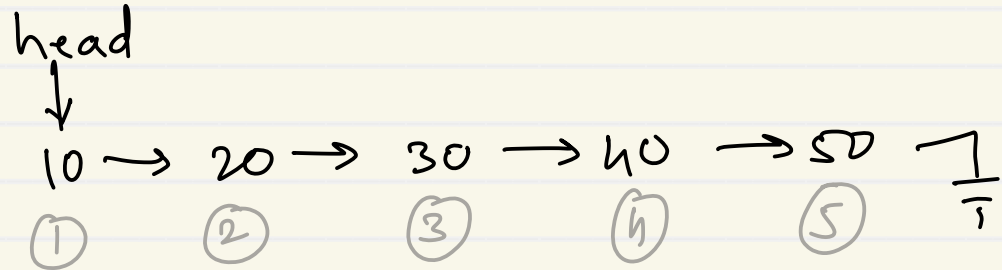
```
Node recReverseList(Node curr) {
    if (curr.next == null) {
        head = curr;
        return curr;
    }
    Node last = recReverseList(curr.next);
    last.next = curr;
    curr.next = null;
    return curr;
}
```

$$T(n) = O(n)$$

$$S(n) = O(n)$$

Linked List - competitive/interview question

- Find middle of singly linear linked list



count = 5	trav	i	i < count/2
	10	0	T
	20	1	T
	30	2	F

to count nodes : n itrs
 to find mid : $n/2$ itrs

 itrs = $\frac{3}{2}n$

$$T \propto \frac{3}{2}n$$

$$T(n) = O(n)$$

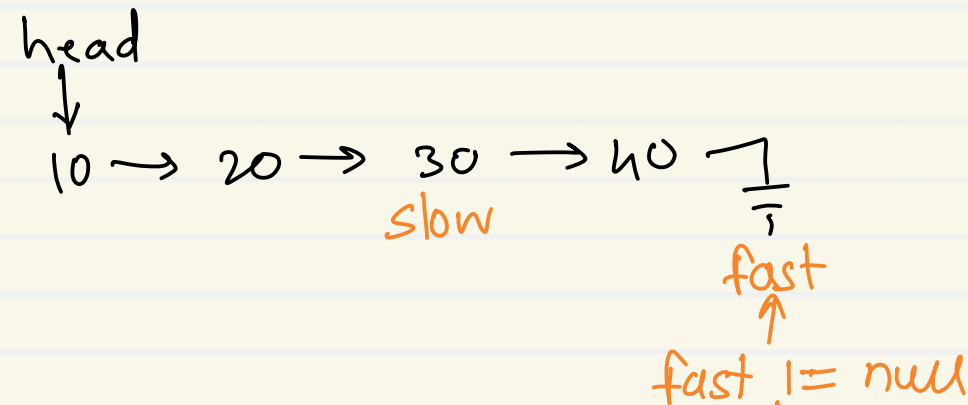
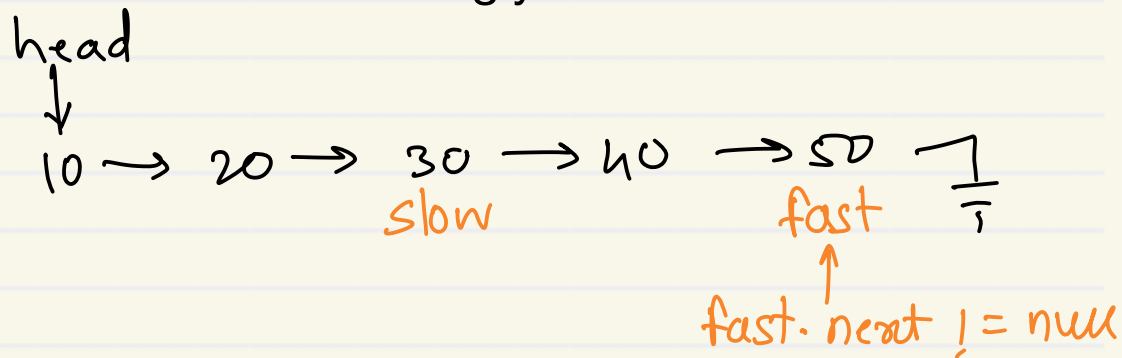
```
Node findMid( Node head) {
```

```
    int count = 0;
    Node trav = head;
    while(trav != null) {
        count++;
        trav = trav.next;
    }
```

```
    Node trav = head;
    for( i=0; i < count/2; i++)
        trav = trav.next;
    return trav;
}
```

$$S(n) = O(1)$$

- Find middle of singly linear linked list



```

Node findMid( ) {
    Node fast = head, slow = head;
    while (fast != null && fast.next != null) {
        fast = fast.next.next;
        slow = slow.next;
    }
    return slow;
}
  
```

$$S(n) = O(1)$$

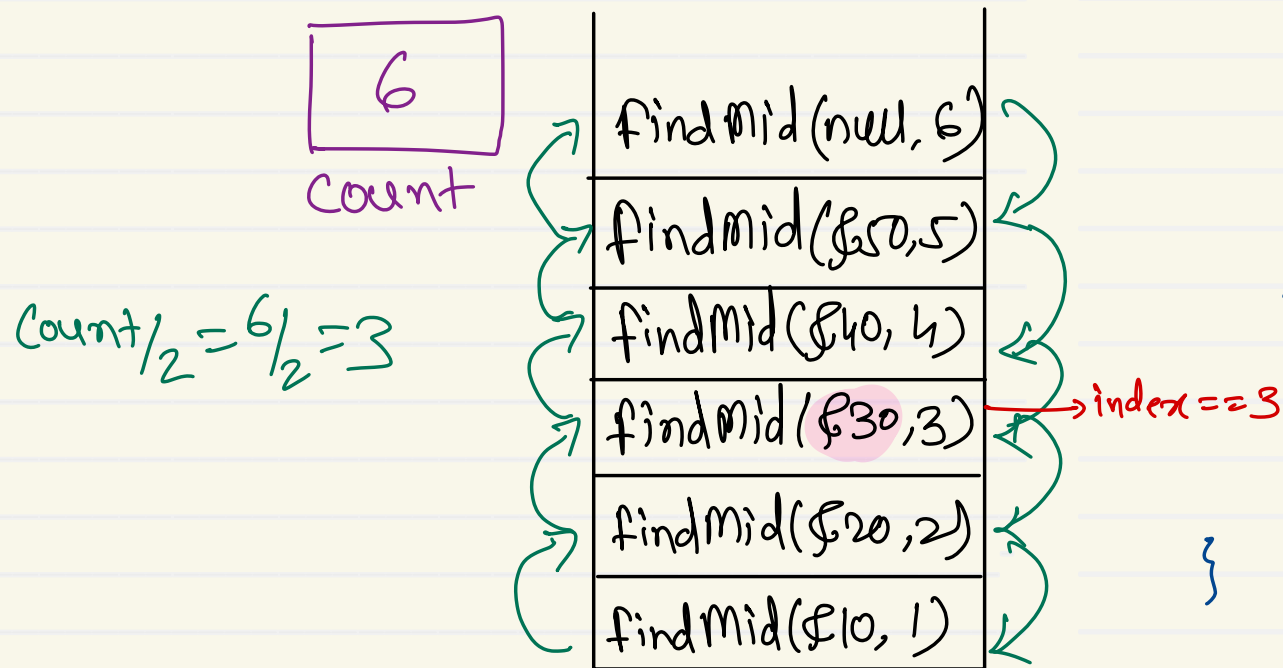
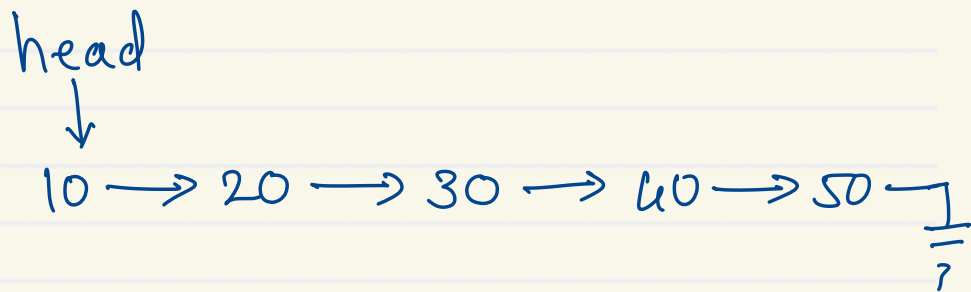
$$itr = n/2$$

$$T \propto \frac{1}{2} n$$

$$T(n) = O(n)$$

Linked List - competitive/interview question

- Find middle of singly linear linked list



```
int count;
```

```
findMid(head, 1);
```

```
Node findMid(Node trav, int index) {
    if (trav == null) {
        count = index;
        return null;
    }
```

```
Node ret = findMid(trav.next, index+1);
```

```
if (index == count/2)
    return trav;
```

```
return ret;
```

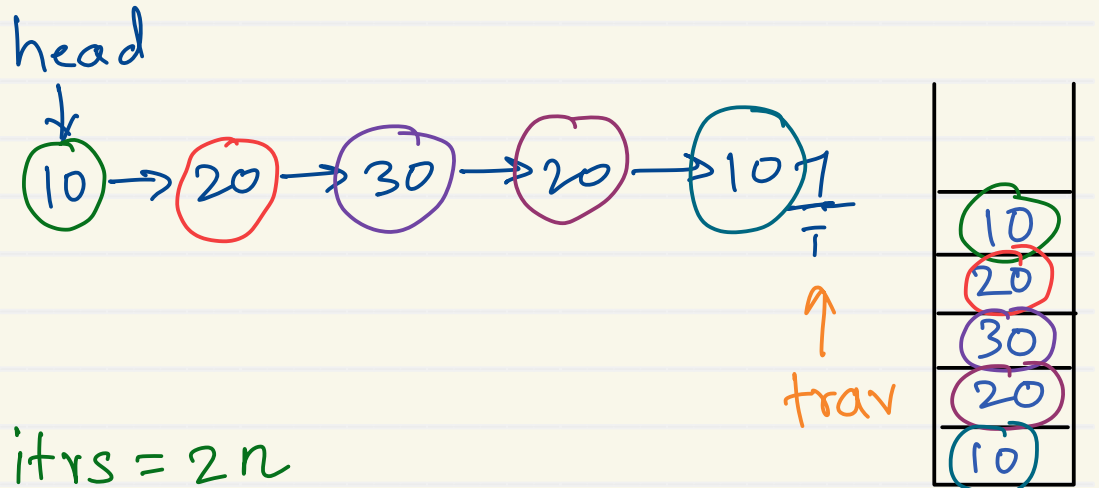
```
}
```

$T(n) = O(n)$

$SC(n) = O(1)$

- Check if linked list is palindrome

1. traverse list & push all nodes on stack one by one
2. pop elements from stack & compare with list elements



$$itrs = 2n$$

$$T \propto 2n$$

$$T(n) = O(n)$$

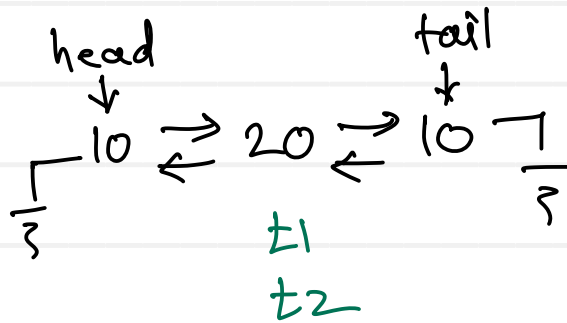
stack

Aux space

$$S(n) = O(n)$$

```
boolean isPalindrome(Node head) {
    Stack<Integer> st = new Stack<>();
    Node trav = head;
    while (trav != null) {
        st.push(trav.data);
        trav = trav.next;
    }
    Node trav = head;
    while (!st.isEmpty()) {
        if (trav.data != st.pop())
            return false;
        trav = trav.next;
    }
    return true;
}
```

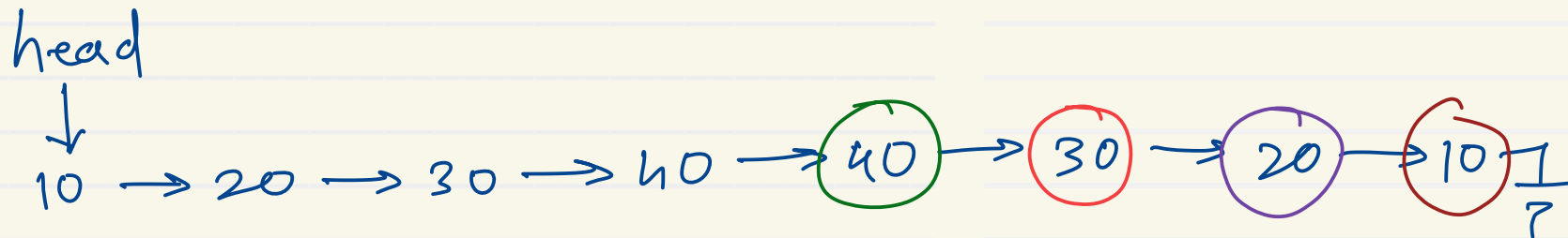
Linked list - Check palindrome



$T(n) = O(n)$
 $S(n) = O(1)$

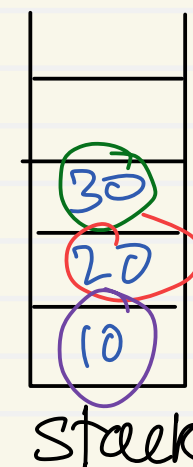
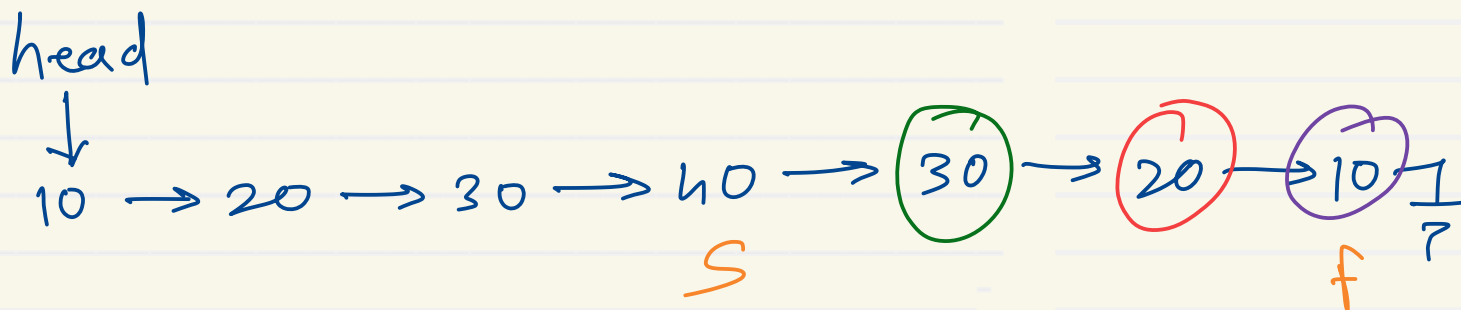
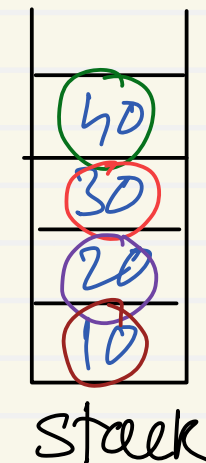
```
boolean isPalindrome ( ) {  
    Node t1 = head, t2 = tail;  
    while (t1 != null && t2 != null) {  
        if (t1.data != t2.data)  
            return false;  
        t1 = t1.next;  
        t2 = t2.prev;  
        if (t1 == t2)  
            return true;  
    }  
    return true;  
}
```

- Check if linked list is palindrome

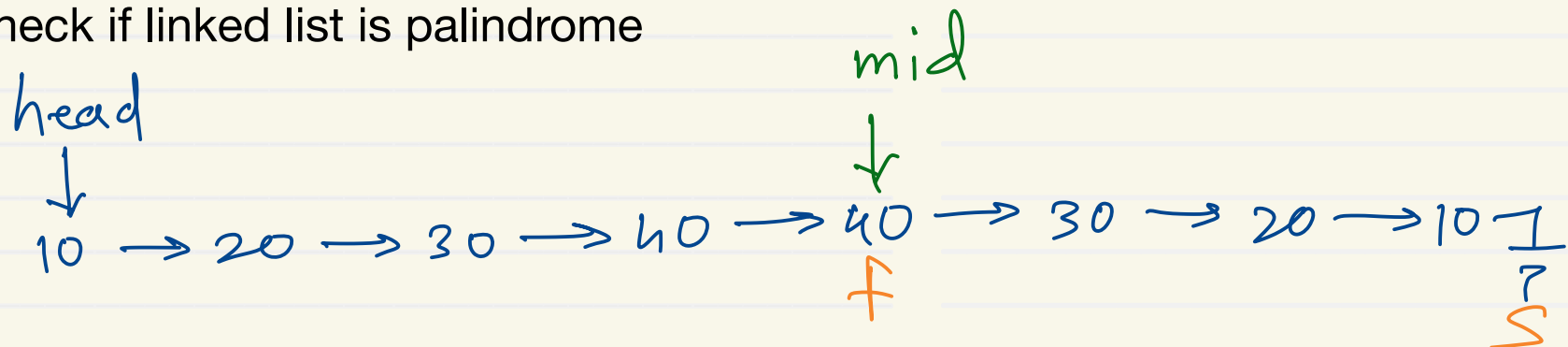


$$itrs = n/2 + n/2 = n \rightarrow T(N) = O(n)$$

$$aux\ space \propto n/2 \rightarrow S(N) = O(N)$$

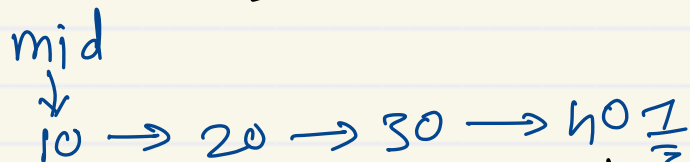


- Check if linked list is palindrome



Step 1 : find mid (use fast & slow ptrs)

Step 2 : reverse 2nd half of the list



Step 3 : compare 1st half & 2nd half of the list,
(head) (mid)

if same palindrome.

Step 4 : reverse 2nd half of list again and append to first half $\rightarrow$ to get original list

$$S(n) = O(1)$$

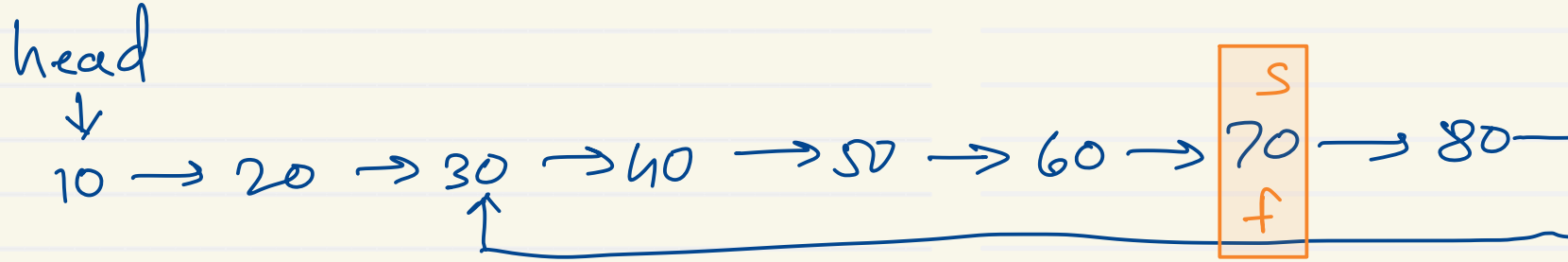
$$\text{itrs} = \frac{n}{2} + \frac{n}{2} + \frac{n}{2} + \frac{n}{2}$$

$$= \frac{4}{2}n = 2n$$

$$T \propto 2n$$

$$T(n) = O(n)$$

- Check if linked list has a loop



```
boolean hasLoop() {  
    Node fast = head, slow = head;  
    while (fast != null && fast.next != null) {  
        fast = fast.next.next;  
        slow = slow.next;  
        if (fast == slow)  
            return true;  
    }  
    return false;  
}
```

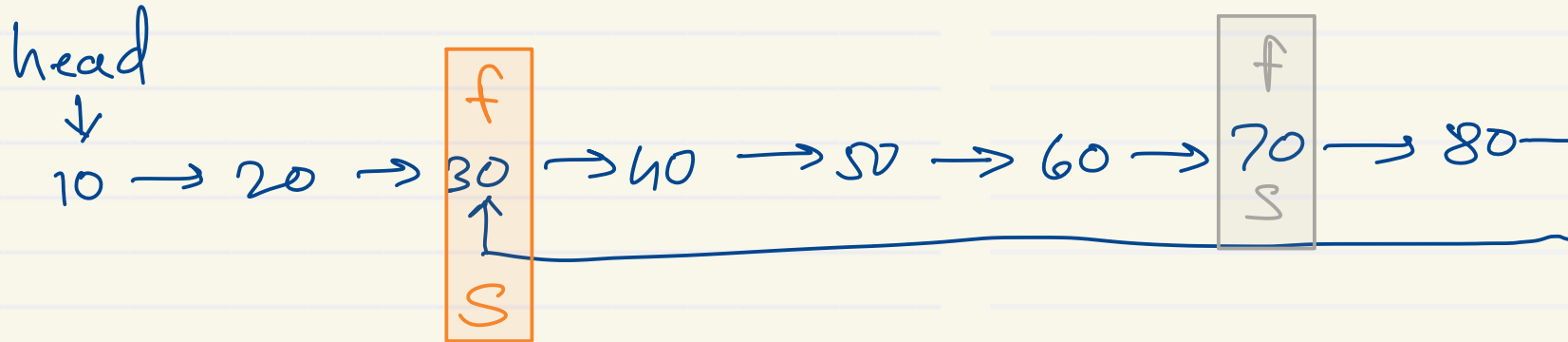
$$S(n) = O(1)$$

$$\text{itrs} = n/2$$
$$T \propto \frac{1}{2} n$$

$$T(n) = O(n)$$

- Check if linked list has a loop

Floyd Cyclic Detection algorithm



- when first time fast & slow becomes equal, cycle is detected.
- keep fast at same place & move slow to head
- now move both with same speed.
- fast & slow will be equal again, which is starting point of the loop.